Review LI.FI Intents August 2025

Overview

About

This review is performed by Gerard Persoon

Scope

The scope for this review is the input part of the LI.FI and OpenIntentsFramework intent system.
The reviewed code is stored in the following repositories:

- https://github.com/openintentsframework/oif-contracts/tree/7370373bbbbe2bcde1e70fca6554107162058522
- https://github.com/catalystsystem/catalyst-intent/tree/7d6a8e720b8418ffa888f71c83d8c5e60528b019

Summary

The review took place between Augusy 4 and August 12, 2025, which included a fix review period.
The analysis resulted in a total of 18 isssues. They can be subdivided as follows:

- 3 vulnerabilities with a risk rating in the category of MEDIUM severity;
- 2 vulnerabilities with a risk rating in the category of LOW severity.

Additionally, 13 informational issues have been found.
The issues have been addressed by the LI.FI project team. 5 issues are acknowledged, 12 issues are solved and 1 issue is partly solved.

Disclaimer

Although the utmost effort has been put in this review, it is no guarantee that all issues have been found and corrected.

Summary of issues

Medium

M-1 : Finalised orders can still be sold
M-2 : `purchaseOrder()` functions don't check `originChainId`
M-3 : Function `open()` doesn't do `_validateInputChain()`

Low

L-1 : `openForAndFinalise()` doesn't use `_purchaseGetOrderOwner()`
L-2 : No check on `destination == 0` in `openForAndFinalise()`

Informational

I-1 : `openForAndFinalise()` doesn't do emits
I-2 : Checks in `_validateFillsNow()` could be made stronger
I-3 : Use of `EfficiencyLib.asSanitizedAddress()`
I-4 : Operation on `msg.sender` inside a loop
I-5 : Improvements on `InputSettlerEscrowLIFI::_resolveLock()`
I-6 : Different nonce patterns
I-7 : Inconsistent use of `calldata input`

I-8 : What happens when `fillDeadline == block.timestamp`
I-9 : References to `IERC7683`
I-10 : Failure causes for `OpenFor()`
I-11 : Checks for DirtyBits not obvious
I-12 : `abi.encodeWithSelector()` has no parameter checks
I-13 : Inaccurate comments

Details of issues

Medium

#M-1 : Finalised orders can still be sold

Description:
There is no explicit check that prevents finalised orders from being sold via `purchaseOrder()`.
Although this is unlikely to happen, it is safer to prevent this.
Recommendation: Consider, in `_purchaseGetOrderOwner()`, to assign a special value to `purchased.purchaser`. This will prevent the execution of `purchaseOrder()`.
For `openForAndFinalise()` it is relevant to have a seperate check because it doesn't use `_purchaseGetOrderOwner()`: in `InputSettlerEscrow::purchaseOrder()` consider checking `orderStatus[] == OrderStatus.Deposited`.
Note: `orderStatus[]` could also be implemented for `InputSettlerCompact`.
LI.FI: Solved for `Escrow` in oif-contracts PR 104. Won't fix on `TheCompact`. This would require setting another storage variable making swaps 20k more expensive.
Reviewer: Verified

#M-2 : `purchaseOrder()` functions don't check `originChainId`

Description:
The functions `InputSettlerCompact::purchaseOrder()` and `InputSettlerEscrow::purchaseOrder()` don't verify that the current chain is the `order.originChainId()`. This is important for the following reasons:

- `_purchaseGetOrderOwner()` uses `purchasedOrders[][]` which is set by `_purchaseOrder()` so both functions should be done on the same chain;
- `_purchaseOrder()` transfers `input` tokens, which are defined on the `originChain`. Other chains could potentially have tokens with the same address but a different implementation;
- orders should be sold only on one chain, to prevent selling the same order multiple times.

Although its unlikely that orders would be sold/bought on the wrong chain, its safer to check for this.
Recommendation: Consider adding `_validateInputChain(order.originChainId())` to the `purchaseOrder()` functions, or to the underlying `_purchaseOrder()` function.
LI.FI: Solved in oif-contracts PR 105.
Reviewer: Verified

#M-3 : Function `open()` doesn't do `_validateInputChain()`

Description:
Function `openFor()`, checks the `InputChain`. However function `open()` doesn't have this:

```
function openFor(bytes calldata order, address sponsor, bytes calldata signature) external {
    // Validate the order structure.
    _validateInputChain(order.originChainId());
    _validateTimestampHasNotPassed(order.fillDeadline());
    ...
}
```

If `open()` is accidentally done on the wrong chain, then the supplied tokens would be lost, because `finalise()` can't be done on the wrong chain.
Recommendation: Consider changing the code to:

```
function open(bytes calldata order) external {
    // Validate the order structure.
+   _validateInputChain(order.originChainId());
    _validateTimestampHasNotPassed(order.fillDeadline());
    ...
}
```

LI.FI: Solved in oif-contracts PR 102.
Reviewer: Verified

Low

#L-1 : `openForAndFinalise()` doesn't use `_purchaseGetOrderOwner()`

Description:
The function `finalise()` and `finaliseWithSignature()` check `_purchaseGetOrderOwner()`.
However the similar function `openForAndFinalise()` doesn't do this.
In theory the order could be also sold via `purchaseOrder()`.
Recommendation: Consider checking `_purchaseGetOrderOwner()` in `openForAndFinalise()`.
Alternatively function `purchaseOrder()` could check that `open()`/`openFor()` has been done.
LI.FI: Solved in oif-contracts PR 104.
Reviewer: Verified

#L-2 : No check on `destination == 0` in `openForAndFinalise()`

Description: Function `finalise()` checks `destination` isn't zero via `_validateDestination()`, however function `openForAndFinalise()` doesn't do that, which is inconsistent.

```
function finalise(...) ... {
    _validateDestination(destination);
    ...
}
```

```
function openForAndFinalise(...) ... {
    // no check on destination
    ...
}
```

```
function _validateDestination(bytes32 destination) internal pure {
    bool isZero;
    // Check if the rigthmost 20 bytes are not all 0. That is a stronger check than the entire 32 bytes.
    assembly ("memory-safe") {
        isZero := iszero(shl(96, destination))
    }
    if (isZero) revert NoDestination();
}
```

Recommendation: Consider also checking `destination == 0` in `openForAndFinalise()`.
LI.FI: Solved in catalyst-intent PR 115.
Reviewer: Verified

Informational

#I-1 : `openForAndFinalise()` doesn't do emits

Description: The function `openForAndFinalise()` doesn't do any emits. This makes tracking the operation via blockexplorers / offchain indexers more difficult.
However the compareable functions `open()` / `openFor()` / `Finalise()` have the emits:

- emit Open(orderId, order);
- emit Finalised(orderId, solver, destination);

Recommendation: Consider adding an emit to `openForAndFinalise()`.

LI.FI: Solved in catalyst-intent PR 114.
Reviewer: Verified

#I-2 : Checks in `_validateFillsNow()` could be made stronger

Description:
Function `_validateFillsNow()` is only used for same chain intents.
Due to this some checks could be made stronger.
Recommendation: Consider checking `output.chainId == block.chainid`.
LI.FI: Acknowledged. The timestamp would enforce this strongly enough.
Reviewer: Verified

#I-3 : Use of `EfficiencyLib.asSanitizedAddress()`

Description:
`EfficiencyLib.asSanitizedAddress()` is used in both InputSettlerEscrow and InputSettlerEscrowLIFI. However this library is part of `The-Compact` which creates an external dependancy which isn't necessary.
Recommendation: Consider using `LibAddress::fromIdentifier()` instead.
LI.FI: Solved in oif-contracts PR 97 and catalyst-intent PR 116.
Reviewer: Verified

#I-4 : Operation on `msg.sender` inside a loop

Description: Function `_validateFillsNow()` applied a operation on `msg.sender` inside a loop. This could be enhanced to improve readability.

```
function _validateFillsNow(...) ... {
    ...
    bytes32 payloadHash = _proofPayloadHash(..., msg.sender.toIdentifier(), ...);
    ...
}
```

Recommendation: Consider calculating `msg.sender.toIdentifier()` outside of the loop. Note: this will cost more gas.
LI.FI: Won't fix. The operation is purely casting and it also makes potential stack issues harder to fix later.
Reviewer: Acknowledged

#I-5 : Improvements on `InputSettlerEscrowLIFI::_resolveLock()`

Description:
Some improvements on `InputSettlerCompactLIFI::_resolveLock()` have not been applied to `InputSettlerEscrowLIFI::_resolveLock()`.
Recommendation: Consider changing the code to:

```
function _resolveLock(...) ... {
    ...
+   address _owner = owner();
-   uint256 fee = governanceFee;
+   uint64 fee = _owner != address(0) ? governanceFee : 0;
    ...
-   SafeTransferLib.safeTransfer(token, owner(), calculatedFee);
+   SafeTransferLib.safeTransfer(token, _owner , calculatedFee);
...
}
```

LI.FI: Solved in catalyst-intent PR 117.
Reviewer: Verified

#I-6 : Different nonce patterns

Description: The similar functions `_openForWithPermit2()` and `_openForWithAuthorization` use a different pattern for nonce:
Function `_openForWithPermit2()` uses `order.nonce()`:

```
function _openForWithPermit2(...) ... {
    ...
    ... permitBatch = ISignatureTransfer.PermitBatchTransferFrom({
        ...
        nonce: order.nonce(),
        ...
    });
}
```

While function `_openForWithAuthorization()` uses `orderId`:

```
function _openForWithAuthorization(...,  bytes32 orderId) ... {
    ...
    ....receiveWithAuthorization({
       ...
       nonce: orderId,
       ...
    });
}
```

Recommendation: Consider documenting this difference.
LI.FI: Acknowledged
Reviewer: Acknowledged

#I-7 : Inconsistent use of `calldata input`

Description: In function `_openForWithAuthorization()`, the first part of the code uses `uint256[2] calldata input = inputs[0]`, however the second part of the code doesn't do this.
The first pattern is more readable.

```
function _openForWithAuthorization(...) ... {
    ...
    if (numInputs == 1) {
        uint256[2] calldata input = inputs[0];
        ...
        if (success) return;
    }
    for (uint256 i; i < numInputs; ++i) {
        ...
        IERC3009(EfficiencyLib.asSanitizedAddress(inputs[i][0])).receiveWithAuthorization({
           ...
            value: inputs[i][1],
            ...
        });
    }
}
```

Recommendation: In the second part of the code consider also assigning `input`.

```
for (uint256 i; i < numInputs; ++i) {
+   uint256[2] calldata input = inputs[i];
    ...
}
```

LI.FI: Solved in oif-contracts PR 100.
Reviewer: Verified

#I-8 : What happens when `fillDeadline == block.timestamp`

Description:

fillDeadline vs validBefore

According to erc7683, which has inspired `InputSettlerEscrow`:

```
fillDeadline is: The timestamp by which the order must be filled on the destination chain(s)
```

eip-3009 uses:

```
validBefore The time before which this is valid (unix time)
```

The function `_openForWithAuthorization()` uses `fillDeadline` at the parameter `validBefore`:

```
function _openForWithAuthorization(...) ... {
    ...
    IERC3009(EfficiencyLib.asSanitizedAddress(inputs[i][0])).receiveWithAuthorization({
        ...
        validBefore: fillDeadline,
        ...
    });
}
```

The caller of `_openForWithAuthorization()` checks:

```
_validateTimestampHasNotPassed(order.fillDeadline());
```

Function `_validateTimestampHasNotPassed()` compensates for the difference between `fillDeadline` and `validBefore` by reverting when `block.timestamp == fillDeadline`, so the code will work as expected.

```
function _validateTimestampHasNotPassed(uint32 timestamp) internal view {
    if (block.timestamp >= timestamp) revert TimestampPassed();
}
```

fillDeadline vs block.timestamp

Additionally there is a difference between `_validateFills()` and `openForAndFinalise()` + `_validateFillsNow()`: `_validateFills()` is ok with `fillDeadline == block.timestamp (of the fill moment)`:

```
function _validateFills(...) ... {
    ...
    if (fillDeadline < outputFilledAt) revert FilledTooLate(fillDeadline, outputFilledAt);
    ...
}
```

Function `openForAndFinalise()` (which calls _validateFillsNow()) reverts when `fillDeadline == block.timestamp` due to _validateTimestampHasNotPassed().

```
function openForAndFinalise(...) ... {
    _validateTimestampHasNotPassed(order.fillDeadline());
    ...
     _validateFillsNow(order.inputOracle(), order.outputs(), orderId);
}

function _validateTimestampHasNotPassed(uint32 timestamp) internal view {
    if (block.timestamp >= timestamp) revert TimestampPassed();
}
```

Recommendation: Double check that `_validateFills()` is ok with `fillDeadline == block.timestamp (of the fill moment)` is the intended behaviour.
Consider documenting the difference between `fillDeadline` and `validBefore` at function `_openForWithAuthorization()`. _validateFillsNow(order.inputOracle(), order.outputs(), orderId);
LI.FI: Acknowledged. The validation functions are intended to be overly safe and as such always blocks equality while the fill function is intended to be cautious to ensure fills are finalise-able.
The current case is intended. The usage of fillDeadline to communicate the openDeadline does cause some friction but it is viewed as a fair tradeoff to simplify the order.
Reviewer: Acknowledged

#I-9 : References to `IERC7683`

Description: Contract `InputSettlerEscrow` still has some references to IERC7683` which are not used.
Recommendation: Consider removing the unused references to `IERC7683`:

```
-import { Output, ResolvedCrossChainOrder } from "../../interfaces/IERC7683.sol";
-import { IOriginSettler } from "../../interfaces/IERC7683.sol";
```

LI.FI: Solved in oif-contracts PR 98.
Reviewer: Verified

#I-10 : Failure causes for `OpenFor()`

Description: Contract `InputSettlerEscrow` has this comment:

```
Since tokens are collected on the `::open(For)` call,
it is important to wait for the `::open(For)` call to be final before filling the intent.
```

In case a solver doesn't wait for this he should be aware of the reasons why `OpenFor()` could fail.
Note: this is also important for someone buying an order via `purchaseOrder()`.
Recommendation: Consider documenting the following potential failure situations for `OpenFor()` :

- `PERMIT2` contract not deployed
- multiple inputs of `_openForWithAuthorization()` that use the same token. This will fail because the same `nonce` is used for each input and this will fail if this is applied multiple times on the same input token.

LI.FI: Acknowledged. A previous fix ensured that purchases has to be of opened orders. Additionally, there are a lot of reasons why opening an order may fail, several of which are outside of our control. Documenting these may cause more harm since consumers may assume that the list is exhaustive.
Reviewer: Acknowledged

#I-11 : Checks for DirtyBits not obvious

Description: Function `_openForWithPermit2()` has an explicit check on DirtyBits on input tokens. This isn't used in other places of the code. However the specific reason why its necessary here is not obvious.

```
function _openForWithPermit2(...) ... {
    ...
    // Validate that the input token's 12 leftmost bytes are 0.
    if ((inputToken >> 160) != 0) revert InputTokenHasDirtyBits();
    address token;
    assembly ("memory-safe") {
        // No dirty bits exist.
        token := inputToken
    }
    ...
}
```

Note: assembly is used, which could also be replaced by function calls.
Recommendation: Conder adding the following as a comment:

```
The reason it is done is because if there are dirty bits it interferes with orderId specification and the
permit2 signature: You open the intent with an unexpected orderId because the excess will get
pruned on the permit2 sig.
To ensure there is a 1:1 map of signed Permit2 message to order, it has to be explicitly checked.
```

Consider replacing the assembly with one of the functions: `LibAddress::fromIdentifier()` or `EfficiencyLib.asSanitizedAddress()`.
LI.FI: Solved in oif-contracts PR 99.
Reviewer: Verified

#I-12 : `abi.encodeWithSelector()` has no parameter checks

Description: Function `_openForWithAuthorization()` uses `abi.encodeWithSelector()` which has no checks on the supplied parameters, which means errors are not detected during compile time.
Recommendation: Consider using `abi.encodeCall()`.
LI.FI: Solved in oif-contracts PR 101.
Reviewer: Verified

#I-13 : Inaccurate comments

Description: Several comments are inaccurate.
Recommendation: Consider changing the comments in the following way:

Incomplete comments

InputSettlerEscrow.sol#L142-L143:

```
-@notice .. tokens are collected from `sponsor` through either permit2 or ERC3009.
+@notice .. tokens are collected from `sponsor` through transferFrom, permit2 or ERC3009.
```

InputSettlerPurchase.sol#L47

```
-@dev ... to opt-in to Compact transfers instead of withdrawals.
+@dev ...to opt-in to Compact transfers instead of withdrawals / to verify the owner/purchaser
```

InputSettlerEscrow.sol#L98, InputSettlerEscrow.sol#L125, InputSettlerEscrow.sol#L148:

```
-@param order  bytes representing ...
+@param order  bytes representing ..., encoded via abi.encode()
```

InputSettlerEscrowLIFI.sol#L125):

```
-The caller ... solvers array.
+The caller ... solvers array or the orderOwner if the order has been purchased.
```

Inaccurate comments

InputSettlerEscrow.sol#L150:

```
-@param signature Allowance signature from user with a signature type encoded as:
+@param signature Allowance signature from sponsor with a signature type encoded as:
```

InputSettlerEscrowLIFI.sol#L203:

```
-This is an important check as it is indeed to process external ERC20 transfers.
+This is an important check as it is intended to process external ERC20 transfers.
```

Copy paste errors

InputSettlerBase.sol#L46

```
-@param timestamp The timestamp to validate that it is not less than block.timestamp
+@param timestamp The timestamp to validate that it is at least equal to block.timestamp
```

InputSettlerBase.sol#L76:

```
-@param destination The timestamp to validate that it is not less than block.timestamp
+@param destination Destination of the funds
```

InputSettlerEscrowLIFI.sol#L21:

```
-... functionality of InputSettlerCompact.
+... functionality of InputSettlerEscrow.
```

InputSettlerEscrowLIFI.sol#L199:

```
-//--- The Compact & Resource Locks ---//
+//--- Resource Locks ---//
```

Typos

InputSettlerBase.sol#L82:

```
-// Check if the rigthmost 20 bytes ...
+// Check if the rightmost 20 bytes ...
```

InputSettlerEscrow.sol#L98, InputSettlerEscrow.sol#L125, InputSettlerEscrow.sol#L148:

```
-@param order  bytes representing an encoded StandadrdOrder.
+@param order  bytes representing an encoded StandardOrder.
```

InputSettlerEscrowLIFI.sol#L21, InputSettlerCompactLIFI.sol#L18:

```
-It inheirts all of the functionality ...
+It inherits all of the functionality ...
```

LI.FI: Solved in oif-contracts PR 103 and catalyst-intent PR 118.
Reviewer: Verified